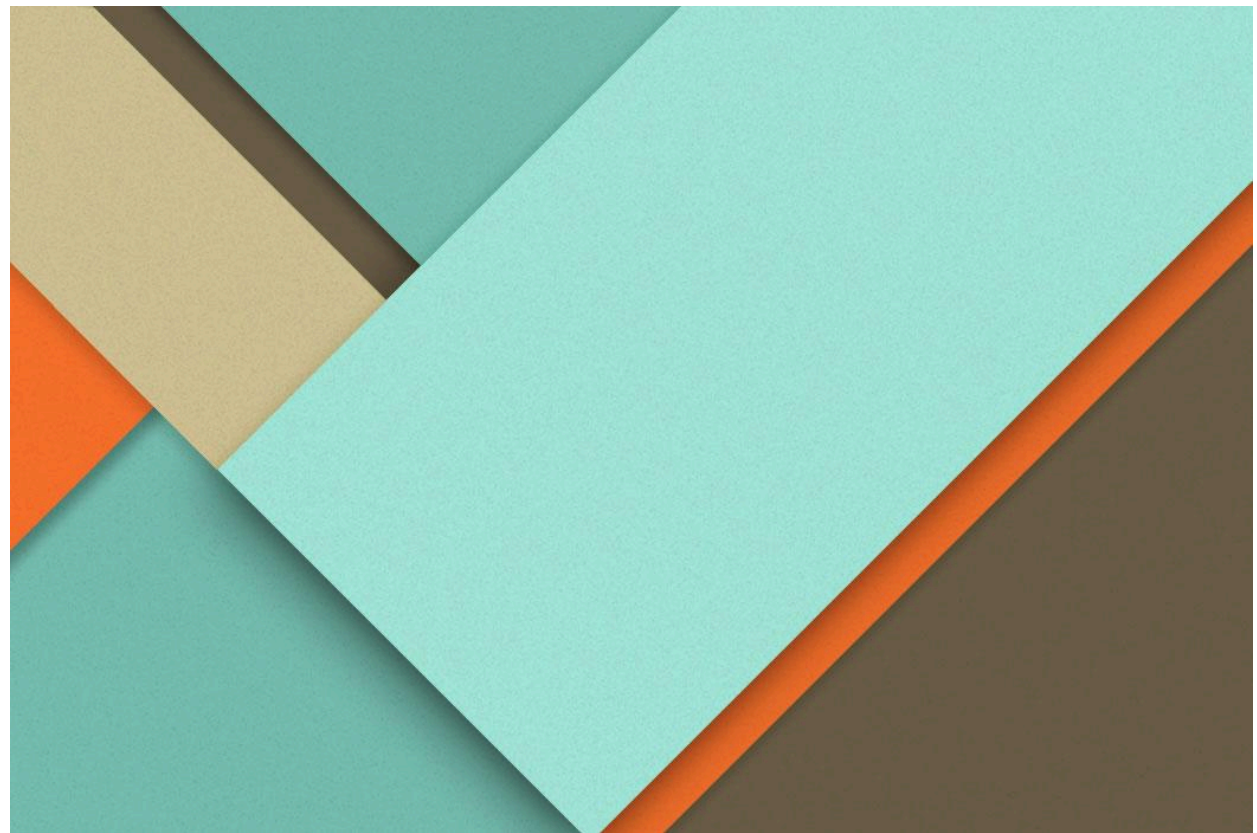




MEMORIA DE PRÁCTICAS MANTENIMIENTO Y GESTIÓN DEL CAMBIO EN SISTEMAS SOFTWARE

Adrián Segura Colino 54791921X

Jaime Abad Quirós 49733043M

Fernando Carrero Pérez 49395551S

ÍNDICE

1. Descripción general del proyecto	2
2. Decisiones de arquitectura	4
3. Desarrollo por entregas	5
3.1. Entrega 1: Configuración del Entorno y Arquitectura Base.....	5
3.2. Entrega 2: Modelo de Dominio y Repositorios (JPA)	6
3.3. Entrega 3: Lógica de Negocio y Servicios	7
3.4. Entrega 4: Máquina de Estados y Trazabilidad	7
3.5. Entrega 5: API REST, Documentación y Containerización	8
3.6. Entrega Final: Release Management y Automatización del Despliegue	9
4. Pipeline CI/CD	10
5. Métricas de calidad (SonarCloud)	11
6. Instrucciones de instalación	12
7. Análisis de deuda técnica	12

1. Descripción general del proyecto

Contexto y Propósito del Sistema

El proyecto consiste en el diseño, desarrollo y evolución de **MGCSS-Track**, una plataforma avanzada para la gestión de solicitudes de servicios técnicos. El sistema nace de la necesidad real de una organización que presta servicios de soporte informático, reparaciones y mantenimiento. Hasta la fecha, dicha entidad gestiona sus flujos de trabajo de manera informal, requiriendo una transición hacia un ecosistema digital que permita registrar solicitudes de forma eficiente, asignarlas a los técnicos competentes, controlar en tiempo real el estado de las tareas, evaluar los tiempos de resolución y extraer métricas de rendimiento básicas.

A diferencia de los desarrollos convencionales centrados exclusivamente en acumular funcionalidad, el pilar fundamental de **MGCSS-Track** ha sido el diseño proactivo bajo estrictos criterios de calidad, mantenibilidad y tolerancia al cambio desde la primera línea de código.

Restricciones Arquitectónicas

El desarrollo del proyecto está sujeto a un conjunto de directrices estructurales e innegociables:

- **Arquitectura por capas o hexagonal:** El sistema garantiza una separación clara entre las capas de Dominio, Servicios, Infraestructura y API.
- **Pruebas y Automatización:** Es obligatorio el uso de pruebas automatizadas y la disponibilidad de un pipeline de Integración Continua (CI/CD) funcional desde el inicio del ciclo de vida del software.

Alcance Funcional Inicial (Versión 0.1)

El núcleo operativo de la plataforma se divide inicialmente en tres grandes módulos funcionales:

- **Gestión de Clientes:** Permite las operaciones de crear, modificar datos básicos, consultar y listar clientes. Los campos mínimos requeridos son id, nombre, email y tipoCliente (clasificado como STANDARD o PREMIUM).

- **Gestión de Técnicos:** Permite crear técnicos, activarlos o desactivarlos de la plataforma y listar los técnicos disponibles. Los campos mínimos contemplados son id, nombre, especialidad y un booleano para indicar si está activo.
- **Gestión de Solicitudes (Entidad Core):** Representa el eje central del negocio para registrar una petición de servicio. Contiene obligatoriamente los campos id, cliente, descripción, fechaCreacion, estado (restringido a ABIERTA, EN_PROCESO, CERRADA), técnicoAsignado (opcional) y fechaCierre (definido como nullable).

Reglas de Negocio Iniciales

El comportamiento del dominio está regulado por restricciones validadas mediante pruebas unitarias:

- Solo se permite asignar un técnico a una solicitud si este se encuentra activo.
- Solo es posible cerrar una solicitud si su estado previo es EN_PROCESO.

Evolución Planificada del Sistema

El proyecto está diseñado para experimentar una evolución controlada a lo largo de su desarrollo incorporando nuevos requisitos técnicos y funcionales:

- **Gestión del Cambio (Cambio 1):** Introducción de la regla de negocio que permite reabrir solicitudes previamente cerradas, obligando a mantener un historial de cambios de estado. Asimismo, la prioridad del cliente PREMIUM pasa a influir de forma directa en el tiempo máximo de resolución (SLA). Este cambio fuerza la modificación de la entidad core, la creación de estructuras para el historial, la refactorización de la lógica existente y el ajuste de tests para verificar la regresión.
- **API y Despliegue (Cambio 2):** Transformación del sistema interno en un producto consumible externamente mediante la exposición de endpoints REST completos. Exige la separación estricta entre el modelo de exposición (DTOs) y las entidades internas. Incluye la documentación automática de la API con OpenAPI (Swagger UI), el control con versionado semántico, la generación de una Release estable y la containerización mediante un Dockerfile funcional.

Métricas Obligatorias de Calidad

La evaluación técnica de la solución final se monitoriza de manera continua analizando los siguientes indicadores de calidad de código:

- Cobertura de pruebas automatizadas.

- Complejidad ciclomática de los métodos.
- Identificación de Code Smells y porcentaje de duplicación de código.
- Ratio de Deuda Técnica (Technical Debt Ratio).

2. Decisiones de arquitectura

El diseño arquitectónico de **MGCSS-Track** se basó en la aplicación de patrones probados para maximizar la mantenibilidad, la tolerancia al cambio y la claridad en la separación de responsabilidades, cumpliendo con la restricción de arquitectura por capas. A continuación, se detallan las decisiones clave adoptadas:

- **1. Arquitectura en Capas:**
 - **Descripción:** Se implementó una separación estricta de responsabilidades: **Dominio** (entidades y reglas de negocio puras, sin dependencias de framework), **Servicio** (orquestración de casos de uso con `@Transactional`), **Infraestructura** (implementaciones JPA como `JpaSolicitudRepository`), y **API** (controladores REST + controlador Thymeleaf).
 - **Justificación:** Aísla el impacto de los cambios, permitiendo la modificación de una capa sin afectar a la lógica de negocio central.
- **2. Patrón DTO (Data Transfer Object):**
 - **Descripción:** Clases como `SolicitudRequestDTO` y `SolicitudResponseDTO` separan el modelo de exposición de las entidades JPA internas.
 - **Justificación:** Evita el acoplamiento y protege al sistema contra el *over-posting*.
- **3. Base de Datos Dual (H2/PostgreSQL):**
 - **Descripción:** Se utiliza **H2 en memoria** para desarrollo y tests (inicio rápido, sin dependencias externas) y **PostgreSQL** para producción (configurado en `application.properties`).
 - **Justificación:** El cambio es transparente a nivel de código gracias a JPA/Hibernate, cumpliendo con el principio de agnóstico a la persistencia.
- **4. Testcontainers para Tests de Integración:**
 - **Descripción:** El test `MgcssTrackApplicationTests` levanta un contenedor PostgreSQL real en Docker durante los tests de integración, inyectando la URL dinámicamente mediante `@DynamicPropertySource`.

- **Justificación:** Garantiza que el comportamiento del sistema en tests es idéntico al de producción, validando la configuración real.

3. Desarrollo por entregas

Entrega 1: Configuración del Entorno y Arquitectura Base

Para esta primera entrega nos centramos principalmente en dejar listo el esqueleto del proyecto. Empezamos con una base limpia y bien estructurada en capas para no tener problemas de organización más adelante cuando la aplicación empezara a crecer y tuviéramos que meter los cambios de requisitos.

A partir de la estructura generada, estos fueron los primeros pasos que dimos:

- **Inicialización del proyecto y CI:** Creamos el proyecto base utilizando Spring Boot y Maven (por eso en la raíz del repositorio se pueden ver los ejecutables mvnw y mvnw.cmd). El punto de arranque de nuestra aplicación quedó definido en la clase MgcssTrackApplication.java. Además, para cumplir con los requisitos de calidad desde el día uno, dejamos creado el archivo .github/workflows/ci.yml para que GitHub Actions ejecutara la integración continua.
- **Configuración de la base de datos (H2):** Como en las primeras entregas lo que nos interesaba era probar las cosas rápido en local sin tener que levantar servicios externos, conectamos el proyecto a la base de datos en memoria H2. Toda esta configuración inicial de conexión la metimos en el archivo application.properties (dentro de src/main/resources). De esta forma, lo dejamos ya preparado para que cambiar a PostgreSQL en el futuro sea solo modificar un par de líneas en ese archivo.
- **Estructura de paquetes (Arquitectura en capas):** Para mantener el código ordenado y separar responsabilidades, dividimos el proyecto en paquetes dentro de src/main/java/com/mgcoss. Creamos las carpetas api (controladores), infrastructure (persistencia) y service. Como Git no permite subir carpetas vacías, tuvimos que usar archivos .gitkeep temporalmente.
- **El modelo de Dominio:** Donde sí metimos código real en esta entrega fue en la capa domain. Al ser el núcleo de la aplicación, dejamos ya creadas la entidad principal Solicitud.java y el enumerado EstadoSolicitud.java para empezar a modelar

los estados de las incidencias.

- **Entorno de pruebas:** Por último, dejamos lista la estructura de testing en `src/test/java/com/mgcss`, manteniendo la clase `MgcssTrackApplicationTests.java` para comprobar que el contexto de Spring Boot levantaba correctamente sin errores antes de empezar a picar la lógica de negocio.

Entrega 2: Modelo de Dominio y Repositorios (JPA)

Para esta segunda entrega, una vez que ya teníamos el esqueleto montado, nos pusimos a trasladar los requisitos del negocio a entidades de software reales y conectarlas con la base de datos. El objetivo principal era dejar la persistencia lista y asegurar que la lógica empezara a tomar forma.

En base a la estructura que fuimos ampliando, los avances se dividieron en los siguientes puntos clave:

- **Entidades Core (El corazón del negocio):** Dentro de nuestro paquete `domain`, empezamos a mapear la realidad del problema. Creamos la entidad principal `Tecnico.java` y le añadimos un nuevo enumerado `EstadoTecnico.java` para controlar su disponibilidad. Por supuesto, ampliamos `Solicitud.java` utilizando anotaciones de JPA (`@Entity`, `@Id`, etc.) para que Spring supiera exactamente cómo traducir nuestras clases Java a tablas en la base de datos relacional.
- **Separación de Repositorios :** Para no romper la arquitectura por capas, aplicamos un diseño muy limpio con las bases de datos. En el paquete `domain` definimos las interfaces puras `SolicitudRepository.java` y `TecnicoRepository.java`. Luego, la implementación real de Spring Data JPA la colocamos en la capa de `infrastructure` (creando `JpaSolicitudRepository.java`). Así, nuestro dominio sigue sin depender directamente del framework de base de datos.
- **Capa de Servicios:** Como las entidades no deben contener lógica compleja, empezamos a darle uso al paquete `service` creando la clase `SolicitudService.java`. Aquí es donde empezamos a centralizar las reglas de negocio reales (como comprobar que un técnico está activo antes de asignarle una solicitud).
- **Entornos de Prueba Aislados:** Para no ensuciar la base de datos de desarrollo cuando ejecutáramos los tests, añadimos el archivo `application-test.yml` en

`src/main/resources`. Esto nos permitió configurar un entorno totalmente aislado y exclusivo para la fase de pruebas.

- **Estructura de Testing:** Nos tomamos muy en serio la calidad del código, así que reestructuramos la carpeta `src/test/java/com/mgcss` en dos subpaquetes para no mezclar conceptos:
 - **unit:** Donde añadimos `SolicitudTest.java`, `TecnicoTest.java` y `SolicitudServiceTest.java` para probar la lógica de las entidades y el servicio de forma aislada y ultrarrápida.
 - **integration:** Donde creamos `SolicitudRepositoryTest.java` para comprobar que las consultas de Spring Data JPA realmente funcionaban contra la base de datos levantada en el entorno de test.

Entrega 3: Lógica de Negocio y Servicios

En esta etapa se construyó el "cerebro" de la aplicación, aislando las reglas de negocio de la capa de acceso a datos.

- **Capa de Servicio:** Implementación de `SolicitudService` como punto de entrada para los casos de uso (crear, buscar, listar).
- **Gestión de Transacciones:** Uso de la anotación `@Transactional` en operaciones de escritura para garantizar la integridad de la base de datos ante posibles fallos.
- **Reglas de Asignación:** Lógica para vincular correctamente a un `Tecnico` con una `Solicitud`, recuperando previamente ambas entidades de la infraestructura.

Entrega 4: Máquina de Estados y Trazabilidad (Gestión del Cambio)

Esta entrega dotó al dominio de protecciones internas para asegurar que las incidencias siguen un ciclo de vida coherente.

- **Ciclo de Vida de la Solicitud:** Implementación de los estados lógicos (`ABIERTA`, `EN_PROCESO`, `CERRADA`).
- **Protección del Dominio:** Inclusión de lógica defensiva mediante el lanzamiento de excepciones (`IllegalStateException`) si se intenta realizar una transición de estado inválida (por ejemplo, cerrar una solicitud que no ha sido iniciada).
- **Historial de Eventos:** Creación de un sistema de trazabilidad (lista de `historial` dentro de la solicitud) que registra automáticamente la fecha y el estado cada vez que la incidencia sufre un cambio.

- **Funcionalidad de Reapertura:** Habilitación lógica para devolver una solicitud CERRADA a EN_PROCESO si la incidencia persiste.

Entrega 5: API REST, Documentación y Containerización (Despliegue)

La entrega final consistió en exponer el sistema al exterior de forma segura, documentada y desplegable en cualquier entorno.

- **Diseño del Contrato (Patrón DTO):** Creación de `SolicitudRequestDTO` y `SolicitudResponseDTO`. Se eliminó la exposición directa de las entidades JPA hacia el cliente para evitar acoplamiento y vulnerabilidades de seguridad.
- **Controladores REST:** Implementación de `SolicitudController` respetando la semántica de los verbos HTTP:
 - POST para creación.
 - GET para consultas individuales y listados.
 - PUT para el cierre (modificación total del estado final).
 - PATCH para la reapertura (modificación parcial).
- **Testing de la Capa Web:** Uso de `@WebMvcTest` y `MockMvc` junto con `Mockito` para validar la respuesta JSON y los códigos de estado HTTP (200 OK, 400 Bad Request) aislando la base de datos.
- **Documentación Interactiva (OpenAPI/Swagger):** Integración de `springdoc-openapi` para autogenerar la interfaz de pruebas en `/swagger-ui.html`. Se documentó la API mediante anotaciones `@Operation` y `@Schema` para facilitar su consumo por terceros.
- **Evidencias de Casos de Uso:** Creación del archivo `use-cases.md` para probar empíricamente (vía Swagger) que la lógica de negocio y las transiciones de estado funcionan correctamente a través de la API.
- **Empaquetado y Dockerización:** * Compilación de la aplicación en un artefacto ejecutable inmutable (`.jar`) mediante Maven.
 - Creación del `Dockerfile` utilizando una imagen base ligera (`eclipse-temurin:17-jre-alpine`) optimizada para producción.
- **Gestión de Versiones (Release):** Uso de flujos Git profesionales mediante Pull Requests para integrar el código en la rama `main`, culminando con el etiquetado semántico (`v1.0.0`) y la publicación oficial de la Release en GitHub.

Entrega Final: Release Management y Automatización del Despliegue

El objetivo de esta última entrega fue profesionalizar el proceso de entrega del software, garantizando que cada versión publicada sea trazable, reproducible y haya pasado obligatoriamente por el Quality Gate antes de ser liberada.

- **Revisión del versionado semántico:** Se analizaron los commits del proyecto aplicando el formato MAJOR.MINOR.PATCH para determinar el impacto de cada cambio. Se publicaron tres versiones: v1.0.0 (primera release estable con API REST y Docker), v1.0.1 (PATCH con el pipeline de release añadido) y v1.0.2 (PATCH con el Quality Gate integrado en el workflow de release). El razonamiento de cada versión quedó justificado en docs/release-notes.md.
- **Automatización del workflow de release:** Se creó el archivo .github/workflows/release.yml con trigger en push de tags con formato v*. El workflow realiza checkout del código, compila el proyecto con Maven, ejecuta los tests, pasa el análisis de SonarCloud (Quality Gate obligatorio), construye la imagen Docker etiquetada con la versión y publica la Release en GitHub adjuntando el .jar como artefacto mediante el CLI nativo gh release create. Se descartó el uso de la action softprops/action-gh-release por generar un Security Hotspot en SonarCloud..
- **Trazabilidad completa:** Se estableció la cadena completa e ininterrumpida commit → merge en main → tag → push tag → pipeline automático → release → artefacto .jar. Ninguna versión puede publicarse sin pasar por este flujo, eliminando cualquier ambigüedad sobre qué commit generó cada versión.
- **Quality Gate como requisito de release:** El workflow de release falla explícitamente si el Quality Gate de SonarCloud no pasa. Esto garantiza que nunca se libera una versión con cobertura por debajo del 80%, bugs o vulnerabilidades conocidas.
- **Extensiones opcionales implementadas:** De forma adicional a los requisitos obligatorios, se implementó una interfaz web con Thymeleaf accesible en /solicitudes (Bootstrap 5, gestión visual de solicitudes y técnicos) que incluye un **Dashboard de métricas** con contadores de estado en tiempo real y **Métricas SLA automatizadas** (con alertas visuales para incumplimientos de 3 días). Además, se desarrollaron tests de integración con Testcontainers que levantan un contenedor PostgreSQL real, validando que el contexto de Spring Boot arranca correctamente contra una base de datos idéntica a la de producción.

4. Pipeline CI/CD

El proyecto utiliza **GitHub Actions** para asegurar la calidad y automatizar la entrega continua mediante dos workflows clave:

- **WORKFLOW 1 - ci.yml (Integración Continua):** Se dispara en cada Pull Request hacia main y en push a main. Pasos: checkout con fetch-depth 0, configurar Java 17 con caché Maven, ejecutar mvn clean verify (compila + tests + Testcontainers), análisis SonarCloud con el plugin sonar-maven-plugin. Requiere que el Quality Gate de SonarCloud pase ($\geq 80\%$ cobertura en código nuevo, 0 bugs, 0 vulnerabilidades) para que el PR pueda mergearse. Variable de entorno: TESTCONTAINERS_RYUK_DISABLED=true para compatibilidad con el runner de GitHub Actions.
- **WORKFLOW 2 - release.yml (Entrega Continua):** Se dispara únicamente cuando se hace push de un tag con formato v* (ej. v1.0.2). Pasos: checkout, configurar Java 17, ejecutar mvn verify + análisis Sonar (Quality Gate obligatorio antes de publicar), construir imagen Docker etiquetada con la versión, publicar Release en GitHub con el .jar adjunto usando el CLI nativo gh release create. Garantiza que nunca se publica una versión sin pasar los tests y el Quality Gate. Trazabilidad completa: commit → tag → pipeline → release → artefacto .jar.

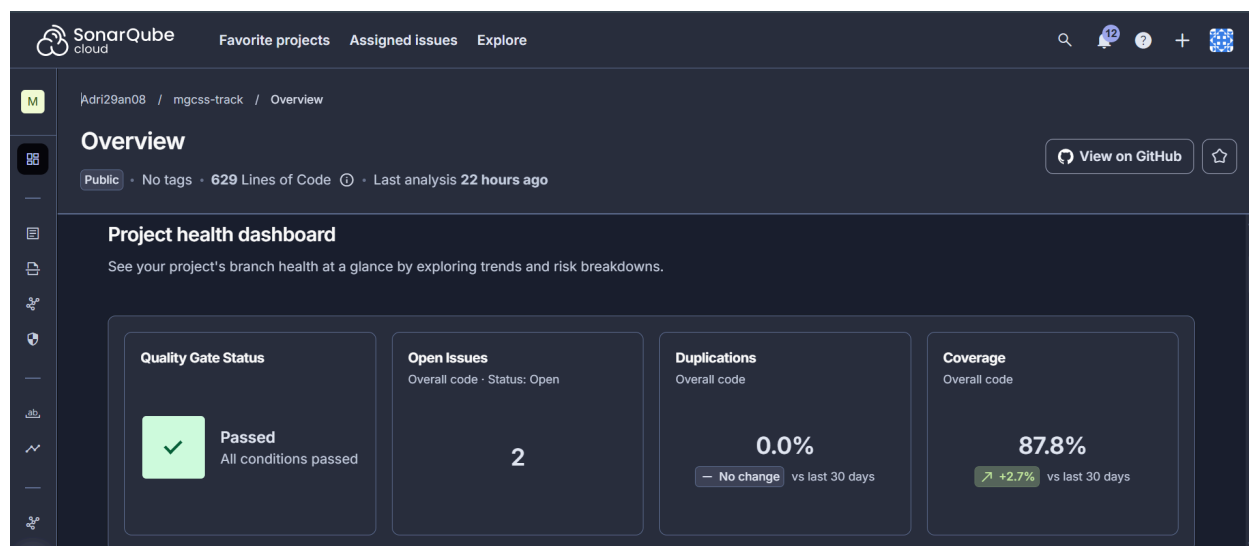
VERSIONADO SEMÁNTICO:

- **v1.0.0:** primera release estable con API REST + Docker.
- **v1.0.1:** PATCH — añadido pipeline release.yml (sin cambios funcionales).
- **v1.0.2:** PATCH — Quality Gate integrado en release.yml.

5. Métricas de calidad (SonarCloud)

Entrega	Coverage	Debt Ratio	Code Smells	Bugs	Quality Gate
Entrega 3	85.5%	0%	0	0	✓ Passed
Entrega 4	92.9%	0%	0	0	✓ Passed
Entrega 5	92.9%	0%	0	0	✓ Passed
Entrega Final	87.8%	0%	2	0	✓ Passed

Las Entregas 1 y 2 no generaron análisis en SonarCloud al estar todavía configurando la infraestructura de CI. A partir de la Entrega 3, todos los merges a main pasaron el Quality Gate obligatorio. El pico de cobertura en las Entregas 4 y 5 (92.9%) refleja el alto ratio de tests sobre código de dominio puro. La ligera bajada en la Entrega Final (87.8%) es esperada: se añadió el controlador Thymeleaf y la integración con Testcontainers, código nuevo que introdujo 2 code smells menores sin impacto en bugs ni en el Quality Gate.



6. Instrucciones de instalación

La versión estable más reciente es v1.0.2, disponible en la página de releases del repositorio de GitHub. Existen dos métodos para ejecutar la aplicación:

Prerrequisitos

- Opción A (JAR): Java 17 o superior instalado en el sistema
- Opción B (Docker): Docker Desktop instalado y en ejecución

Opción A — Ejecución directa con Java

1. Acceder a la página de releases del repositorio en GitHub
2. Descargar el archivo .jar adjunto en la release v1.0.2
3. Abrir una terminal en la carpeta donde se descargó el archivo y ejecutar: `java -jar mgcss-track-0.0.1-SNAPSHOT.jar`

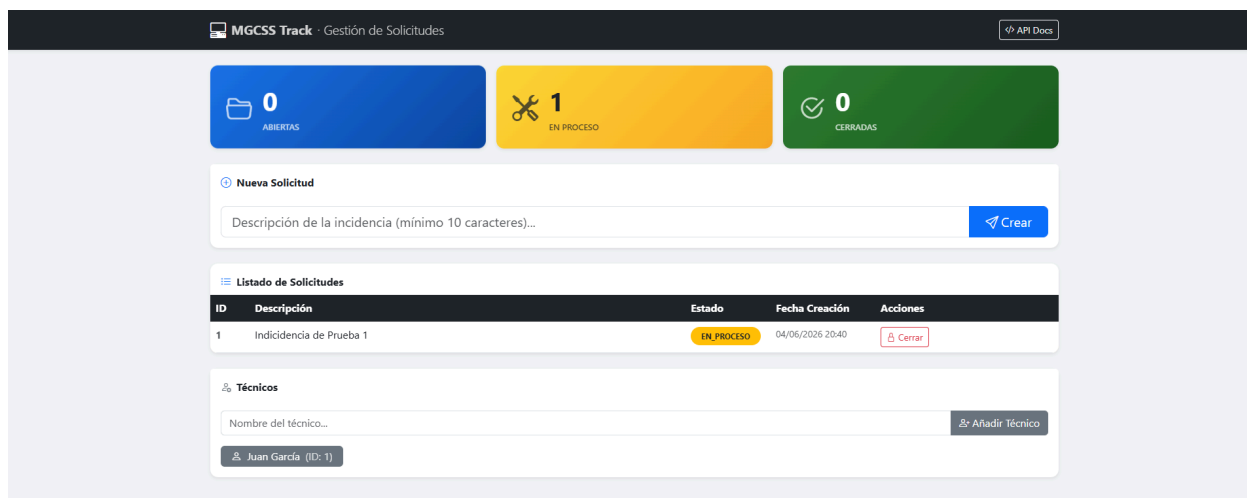
Opción B — Ejecución con Docker

1. Clonar el repositorio o descargar el código fuente de la release v1.0.2
2. Desde la raíz del proyecto, compilar y empaquetar la aplicación: `mvn package -DskipTests`
3. Construir la imagen Docker: `docker build -t mgcss-track .`
4. Ejecutar el contenedor: `docker run -p 8080:8080 mgcss-track`

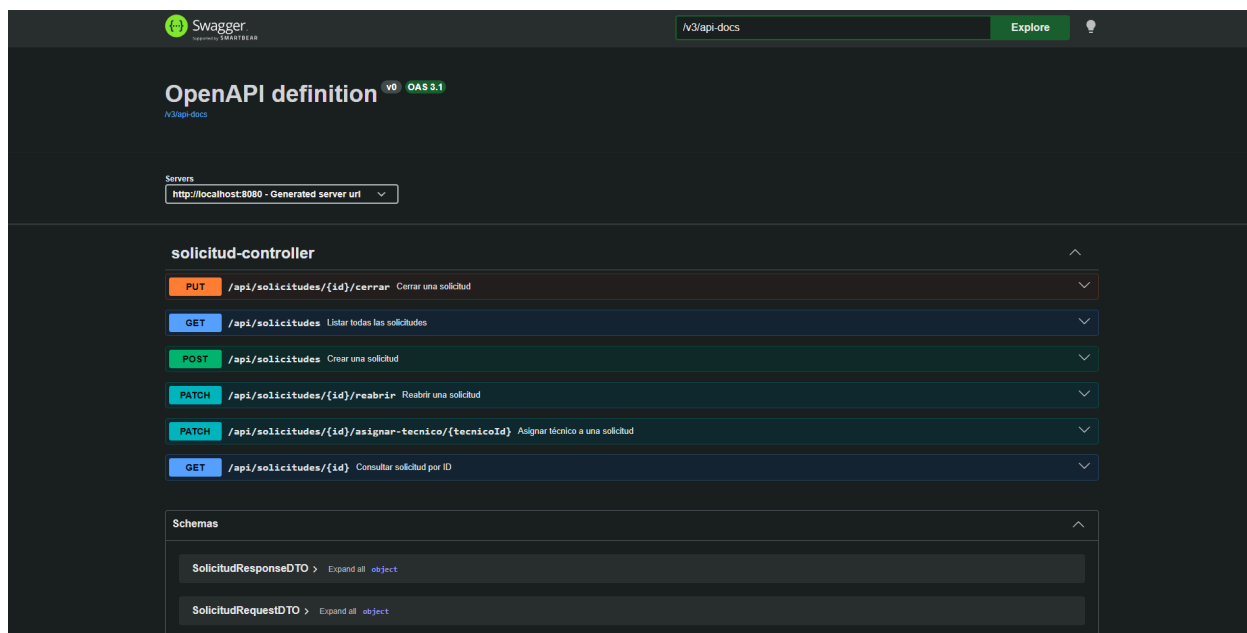
Verificación

Una vez que la aplicación esté en ejecución, hay dos puntos de acceso:

- **Interfaz web:** <http://localhost:8080/solicitudes> Permite gestionar solicitudes e incidencias visualmente, crear técnicos y visualizar estadísticas en tiempo real.



- **API REST (Swagger UI):** <http://localhost:8080/swagger-ui.html> Documentación interactiva donde se pueden probar todos los endpoints directamente desde el navegador.



⚠ Nota: La persistencia opera sobre una base de datos en memoria (H2). Los datos no se conservan al detener la aplicación.

7. Análisis de deuda técnica

A la hora de afrontar el proyecto, tuvimos que ser realistas con los tiempos y priorizar qué hacer. Decidimos sacrificar ciertos puntos del enunciado para asegurarnos de que la arquitectura, el pipeline de CI/CD y el Quality Gate de SonarCloud fueran impecables, en lugar de intentar meter todo con calzador y que el sistema quedara inestable.

Por falta de tiempo, dejamos estos elementos fuera del alcance:

- **Gestión de clientes y campos del técnico:** Aunque el enunciado base los mencionaba, en los *handouts* de las prácticas no se nos exigió implementarlos explícitamente en ninguna entrega. Decidimos dejarlo pasar para centrarnos 100% en el *core* del negocio: la máquina de estados de las solicitudes y la lógica de asignación.
- **Prioridad PREMIUM y reglas de SLA avanzadas:** Al no implementar el módulo de clientes, la regla de negocio que vinculaba el tipo de cliente con el SLA no tenía sentido y se quedó fuera.
- **Extensiones opcionales:** Aunque nos habría encantado hacer más, tuvimos que descartar cosas para poder dedicar ese tiempo a asegurar que la interfaz con Thymeleaf, las métricas del SLA y los tests con Testcontainers funcionaran correctamente.

Básicamente, preferimos tener un sistema robusto y bien probado antes que una lista de funcionalidades incompleta o con deuda técnica oculta. Todo esto lo dejamos anotado como backlog por si el proyecto tuviera continuidad.